

# SLOT1 REACT REDUX

Lập trình full stack là một khái niệm trong phát triển phần mềm, ám chỉ việc một lập trình viên hoặc một nhóm lập trình viên có khả năng làm việc với tất cả các khía cạnh của một ứng dụng phần mềm, bao gồm cả phía **front-end** (giao diện người dùng) và **back-end** (hệ thống xử lý dữ liệu và logic phía máy chủ).

## 1. Các thành phần của lập trình full stack

### a. Front-End

Đây là phần giao diện mà người dùng cuối nhìn thấy và tương tác. Nhiệm vụ chính của lập trình viên front-end là thiết kế và xây dựng giao diện đẹp mắt, thân thiện, dễ sử dụng, và có thể tương tác tốt. Các công nghệ thường được sử dụng bao gồm:

- **HTML (HyperText Markup Language):** Xây dựng cấu trúc trang web.
- **CSS (Cascading Style Sheets):** Tạo kiểu dáng, bố cục và hiệu ứng cho trang web.
- **JavaScript:** Thêm tính năng tương tác, như các nút bấm, menu động, hoặc xử lý sự kiện.
- **Frameworks/Thư viện:**
  - React.js
  - Angular
  - Vue.js

### b. Back-End

Đây là phần chịu trách nhiệm xử lý dữ liệu, logic nghiệp vụ, và quản lý giao tiếp với cơ sở dữ liệu. Công việc của lập trình viên back-end bao gồm:

- Xử lý các yêu cầu từ người dùng (qua front-end).
- Tương tác với cơ sở dữ liệu để lưu trữ hoặc truy xuất dữ liệu.
- Xây dựng các API (Application Programming Interface) để giao tiếp giữa front-end và back-end.

Các ngôn ngữ và công cụ thường được sử dụng:

- **Ngôn ngữ lập trình:** Python, Java, Node.js (JavaScript), Ruby, PHP, C#.
- **Frameworks:** Django, Flask, Express.js, Spring Boot, Ruby on Rails.
- **Cơ sở dữ liệu:** MySQL, PostgreSQL, MongoDB, Redis.

### c. DevOps và quản lý máy chủ

Ngoài front-end và back-end, lập trình viên full stack còn cần hiểu cách triển khai, quản lý và duy trì ứng dụng trên máy chủ. Điều này bao gồm:

- Quản lý máy chủ web (Apache, Nginx).
- Sử dụng container như Docker.
- Triển khai ứng dụng trên các nền tảng như AWS, Azure, Google Cloud, hoặc Heroku.
- CI/CD (Continuous Integration/Continuous Deployment).

## 2. Kỹ năng cần thiết của lập trình viên full stack

Lập trình viên full stack cần sự kết hợp của nhiều kỹ năng:

- **Kỹ thuật:**
  - Thành thạo ít nhất một framework front-end và back-end.
  - Kỹ năng làm việc với cơ sở dữ liệu.
  - Kiến thức về API (RESTful API, GraphQL).
  - Hiểu biết về DevOps và các công cụ triển khai.
- **Thiết kế:** Hiểu biết về thiết kế giao diện (UI/UX).
- **Phân tích:** Hiểu rõ yêu cầu nghiệp vụ và chuyển hóa thành các giải pháp kỹ thuật.

### 3. Lợi ích của lập trình full stack

- **Linh hoạt:** Full stack developer có thể đảm nhận nhiều nhiệm vụ khác nhau trong dự án.
- **Hiệu quả:** Có khả năng quản lý toàn bộ quy trình phát triển ứng dụng.
- **Tiết kiệm:** Trong các nhóm nhỏ hoặc startup, một lập trình viên full stack có thể thay thế cho nhiều vai trò riêng lẻ.

### 4. Hạn chế

- **Khó chuyên sâu:** Vì phải nắm vững nhiều khía cạnh, họ có thể không chuyên sâu bằng các chuyên gia trong từng lĩnh vực cụ thể.
- **Áp lực công việc cao:** Phải giải quyết nhiều loại vấn đề, từ giao diện đến máy chủ.

### 5. Lộ trình để trở thành lập trình viên full stack

1. **Học nền tảng lập trình:**
  - Bắt đầu với các ngôn ngữ cơ bản như JavaScript, Python.
2. **Học front-end:**
  - Thành thạo HTML, CSS, JavaScript.
  - Làm quen với các framework như React.js, Vue.js.
3. **Học back-end:**
  - Nắm vững một ngôn ngữ back-end như Node.js, Python.
  - Học cách làm việc với cơ sở dữ liệu (MySQL, MongoDB).
4. **Học DevOps:**
  - Tìm hiểu về máy chủ, container, và cách triển khai ứng dụng.
5. **Thực hành:** Tham gia các dự án nhỏ, từ thiết kế giao diện đến triển khai back-end.

### 6. Ví dụ về ứng dụng full stack

- Một trang web thương mại điện tử:
  - **Front-end:** Hiển thị sản phẩm, giỏ hàng.
  - **Back-end:** Xử lý thông tin đơn hàng, quản lý người dùng, và lưu trữ dữ liệu sản phẩm.
  - **DevOps:** Triển khai và bảo trì hệ thống.

# THÔNG TIN KHÓA HỌC

## 1. Giới thiệu khóa học:

**Tên khóa học:** Phát triển phía server với NodeJS, Express và MongoDB.

- **Mục tiêu:** Giúp sinh viên làm quen với các công nghệ phía server để xây dựng các ứng dụng web mạnh mẽ và linh hoạt.
  - **Tại sao nên học khóa học này:**
    - Node.js cho phép sử dụng JavaScript trên cả front-end và back-end, giúp tăng hiệu quả phát triển.
    - Node.js được sử dụng rộng rãi bởi các công ty công nghệ lớn như Netflix, PayPal, Uber.
- 

## 2. Điều kiện tiên quyết:

- Hoàn thành các môn học:
    - **DBI202:** Có thể liên quan đến cơ sở dữ liệu cơ bản.
    - **FER202:** Liên quan đến lập trình front-end hoặc nền tảng cơ bản về phát triển ứng dụng web.
- 

## 3. Mục tiêu khóa học:

- Hiểu các khái niệm cơ bản về phát triển phía server, CRUD và REST.
  - Xây dựng và cấu hình một server phía backend bằng NodeJS.
  - Xây dựng API RESTful để kết nối front-end với backend.
- 

## 4. Mô tả khóa học:

- Tìm hiểu giao thức web HTTP, HTTPS.
- Giới thiệu NodeJS và framework Express.
- Sử dụng MongoDB và Mongoose để làm việc với dữ liệu NoSQL.
- Hiểu và triển khai mô hình MVC.
- Tìm hiểu về RESTful API, bảo mật và xác thực web.

### Lợi ích khi hoàn thành:

- Hiểu cách Node.js hoạt động.
  - Xây dựng ứng dụng RESTful API nhanh, mạnh mẽ.
  - Làm việc với dữ liệu NoSQL và công cụ chuyên nghiệp như Mongoose, Express.
  - Xây dựng ứng dụng hoàn chỉnh, tạo nền tảng cho các dự án trong tương lai.
-

## 5. Kế hoạch học tập:

- **Chi tiết kế hoạch học tập:** Có trên FLM (<https://flm.fpt.edu.vn>).
- 

## 6. Tài liệu tham khảo:

- Sách và hướng dẫn về Node.js, Express, MongoDB, EJS, React.
  - Các nguồn trực tuyến như MongoDB, ExpressJS, NodeJS API.
- 

## 7. Môi trường học tập:

- **Công cụ:** NodeJS (v20.12.2), NPM, Express, MongoDB, Postman, Visual Studio Code.
  - **Yêu cầu:** Notebook để ghi chép báo cáo bài tập.
- 

## 8. Quy định lớp học:

- Chuẩn bị bài trước khi đến lớp.
  - Tham gia học tập tích cực, viết báo cáo các bài tập.
  - Không sử dụng điện thoại, game trong giờ học.
- 

## 9. Chiến lược đánh giá:

- **Thành phần đánh giá:**
    - Bài kiểm tra tiến độ (15%).
    - Bài tập (20%).
    - Dự án (15%).
    - Bài kiểm tra thực hành (25%).
    - Bài thi cuối kỳ (25%).
  - **Yêu cầu vượt qua:**
    - Điểm các thành phần  $\geq 0$ .
    - Điểm thi cuối kỳ  $\geq 4$ .
    - Tổng điểm  $\geq 5$ .
- 

## 10. Phương pháp học tập:

- Tập trung lắng nghe và ghi chép.
- Hỏi đáp, đưa ví dụ minh họa.
- Thảo luận nhóm, phân tích, thiết kế, và thực hành.
- Viết báo cáo cho từng bài tập.

---

## 11. Chính sách học thuật:

- **Gian lận:** Không được phép trong thi cử hoặc làm bài.
  - **Đạo văn:** Cần trích dẫn nguồn đầy đủ khi sử dụng nội dung của người khác.
  - **Vi phạm bản quyền:** Sao chép sách mà không được phép là hành vi vi phạm.
- 

## 12. Hỗ trợ:

- Cài đặt các công cụ lập trình khi cần thiết.
- Hỏi đáp trực tiếp hoặc trên diễn đàn.

# GIỚI THIỆU VỀ BACK-END DEVELOPMENT

## 1. Giới thiệu khái niệm cơ bản:

- **Front-End:** Phần giao diện người dùng tương tác, xây dựng bằng HTML, CSS, và JavaScript. Các framework như Bootstrap và JQuery giúp tăng hiệu quả phát triển.
  - **Back-End:** Phần logic và xử lý dữ liệu của website, bao gồm:
    - Xử lý các yêu cầu từ Front-End.
    - Truy xuất dữ liệu từ cơ sở dữ liệu.
    - Đảm bảo hệ thống hoạt động đúng với logic kinh doanh.
  - **Full Stack:** Bao quát cả Front-End và Back-End, thường được các công ty nhỏ hoặc startup ưa chuộng.
- 

## 2. Kỹ năng cần thiết cho lập trình viên Back-End:

- Hiểu biết sâu rộng về nhiều ngôn ngữ lập trình như Python, Java, PHP, SQL, Git, HTML, CSS, JavaScript.
  - Kỹ năng giao tiếp hiệu quả với các nhà phát triển Front-End và đội ngũ thiết kế.
- 

## 3. Công cụ và công nghệ Back-End:

### *Máy chủ (Servers):*

- Cung cấp tài nguyên như lưu trữ file, bảo mật, mã hóa, cơ sở dữ liệu, và các dịch vụ web.
- Có thể đặt tại chỗ (on-premises) hoặc sử dụng máy chủ đám mây (AWS, GCP, Azure).

### *Cơ sở dữ liệu (Databases):*

- **SQL và NoSQL:** SQL phù hợp với dữ liệu có cấu trúc (MySQL, PostgreSQL), NoSQL phù hợp với dữ liệu phi cấu trúc (MongoDB).
  - Quan trọng để chọn cơ sở dữ liệu phù hợp với ứng dụng.
- 

## 4. Cấu trúc dữ liệu và thuật toán:

- Back-End Developer cần tạo mã truy vấn và ánh xạ quan hệ để lấy dữ liệu từ cơ sở dữ liệu.
  - Hiểu cách hoạt động của các hệ quản trị cơ sở dữ liệu như MySQL, MongoDB, Oracle.
- 

## 5. Ngôn ngữ lập trình và framework:

- **JavaScript:** Node.js và Express.js cho server-side.
- **Python:** Django và Flask là các framework phổ biến.
- **Java:** Sử dụng Spring, Java Server Faces.

- **C#:** Một kiến trúc được khuyến nghị.
- **PHP, Perl, Ruby:** Các lựa chọn khác cho Back-End.

*Lưu ý: Hiểu biết cơ bản về công nghệ Front-End (HTML, CSS) cũng hữu ích.*

---

## 6. Kiểm soát phiên bản:

- Sử dụng hệ thống như Git và GitHub để quản lý file dự án, theo dõi lịch sử thay đổi, và hợp tác với nhóm.
- 

## 7. Nền tảng lưu trữ web (Web Hosting Platforms):

- Cung cấp giải pháp lưu trữ sản phẩm trên đám mây, ví dụ: AWS, Google Cloud, Microsoft Azure, Heroku, Firebase.
- 

## 8. Nhiệm vụ của Back-End Developer:

- Thiết kế các giải pháp hiệu quả.
  - Phát triển và quản lý API.
  - Tạo giải pháp lưu trữ và chấp nhận dữ liệu.
  - Viết, kiểm thử, và bảo trì mã nguồn.
  - Debug và xử lý lỗi ứng dụng.
  - Sử dụng các phương pháp phát triển sản phẩm như Agile Scrum.
- 

## 9. Continuous Integration and Continuous Deployment (CI/CD):

- **Continuous Integration (CI):** Tích hợp mã nguồn tự động và thường xuyên vào kho mã.
  - **Continuous Deployment (CD):** Triển khai tự động mã nguồn đã được kiểm tra.
- 

## 10. Tóm tắt nội dung:

- Hiểu Front-End, Back-End, và Full Stack.
- Nắm vững cấu trúc dữ liệu, thuật toán, và framework lập trình.
- Làm quen với web hosting, CI/CD, và các công cụ quản lý mã nguồn.

## 1. Giới thiệu về Node.js

- **Node.js là gì?**
    - Là một môi trường chạy JavaScript trên server, xây dựng trên **Chrome V8 JavaScript Engine**.
    - Hỗ trợ xây dựng các ứng dụng mạng nhanh và mở rộng (scalable), đặc biệt phù hợp cho các ứng dụng thời gian thực, xử lý nhiều dữ liệu.
    - Node.js = Môi trường runtime + Thư viện JavaScript.
  - **Tính năng nổi bật:**
    - **Event-driven và Asynchronous:** Xử lý yêu cầu không đồng bộ, giảm thời gian chờ.
    - **Single-threaded nhưng mở rộng cao:** Có thể xử lý nhiều kết nối đồng thời.
    - **Rất nhanh:** Tận dụng V8 engine của Chrome.
    - **Không đệm:** Dữ liệu được xử lý từng phần thay vì đệm toàn bộ.
- 

## 2. Các ứng dụng của Node.js

- **Nên dùng Node.js:**
    - Ứng dụng I/O (Input/Output) lớn.
    - Ứng dụng stream dữ liệu.
    - Ứng dụng thời gian thực, nặng về dữ liệu.
    - Ứng dụng dựa trên JSON API.
    - Single Page Applications (SPA).
  - **Không nên dùng Node.js:**
    - Ứng dụng cần xử lý nhiều tính toán CPU nặng.
- 

## 3. Kiến trúc Node.js

- **Event Loop:** Là cơ chế xử lý các yêu cầu không đồng bộ.
    - Chia thành các giai đoạn:
      1. **Timers:** Thực thi các callback từ `setTimeout` và `setInterval`.
      2. **Pending Callbacks:** Thực thi các callback hoãn lại.
      3. **Poll:** Xử lý I/O callback và tìm sự kiện mới.
      4. **Check:** Gọi các callback của `setImmediate`.
      5. **Close Callbacks:** Đóng các kết nối, như socket.
  - **Emitter:** Sử dụng sự kiện và callback để xử lý các tác vụ không đồng bộ.
- 

## 4. Cài đặt môi trường Node.js

*Hướng dẫn:*

1. Tải Node.js từ: <https://nodejs.org/>.



## 2. Kiểm tra phiên bản:

- o `node -v`: Kiểm tra phiên bản Node.js.
- o `npm -v`: Kiểm tra phiên bản Node Package Manager (NPM).

## 3. Khởi tạo dự án Node.js:

- o Chạy lệnh `npm init` để tạo file `package.json`.

*NPM (Node Package Manager):*

- **Chức năng:**
    - o Quản lý các module/package của Node.js.
    - o Dễ dàng chia sẻ và cài đặt lại dự án.
  - **Cấu trúc package.json:**
    - o Chứa thông tin về dự án, các dependencies.
- 

# 5. Ứng dụng đầu tiên với Node.js

*Ví dụ: Giải phương trình bậc hai*

## 1. Tạo file `index.js`:

```
javascript
Sao chép mã
const solveQuadraticEquation = (a, b, c) => {
  const delta = b ** 2 - 4 * a * c;
  if (a == 0) return solveLinearEquation(b, c);
  if (delta < 0) return "No real solution";
  if (delta === 0) {
    let x = -b / (2 * a);
    return [x.toFixed(2)];
  } else {
    let x1 = (-b + Math.sqrt(delta)) / (2 * a);
    let x2 = (-b - Math.sqrt(delta)) / (2 * a);
    return [x1.toFixed(2), x2.toFixed(2)];
  }
};

const solveLinearEquation = (a, b) => {
  if (a === 0) return b === 0 ? "Infinitely many solutions" : "No solution";
  let x = -b / a;
  return [x.toFixed(2)];
};

console.log("Quadratic:", solveQuadraticEquation(1, -5, 6));
console.log("Linear:", solveLinearEquation(2, 3));
```

## 2. Chạy ứng dụng:

- o Dùng lệnh `node index.js`.
- 

# 6. Callback, Promise, Async/Await

### Callback:

- Một hàm được truyền vào để xử lý kết quả của tác vụ không đồng bộ.
- Nhược điểm: **Callback Hell**, code khó đọc.

### Promise:

- Làm cho code dễ đọc hơn bằng cách xử lý kết quả không đồng bộ:
  - **Fulfilled**: Thành công.
  - **Rejected**: Thất bại.
  - **Pending**: Đang chờ.

### Async/Await:

- Cách tiếp cận mới để xử lý Promise.
- Code dễ hiểu, mô phỏng như đồng bộ:

```
javascript
Sao chép mã
const fetchData = async () => {
  try {
    const data = await fetch("https://api.example.com");
    console.log(data);
  } catch (error) {
    console.error(error);
  }
};
```

---

## 7. Tổng kết

- Hiểu cơ bản về Node.js và môi trường NPM.
- Thực hành viết ứng dụng đầu tiên.
- Tìm hiểu về Event Loop, callback, Promise và Async/Await.
- Node.js phù hợp với các ứng dụng thời gian thực, API và dữ liệu lớn.